Object Recognition Using Landmark Detection and Camera Calibration Matrix with OpenCV in Python

1. Aim

To develop a Python-based object recognition system using landmark detection, feature matching, and the camera calibration matrix with the OpenCV package.

2. Problem Statement

Develop an object recognition application that identifies a known object in a camera image using visual landmarks and camera calibration parameters.

The system should detect distinctive landmarks in a reference image, match them with landmarks in a query image, and use the camera calibration matrix to correct lens distortion and improve image accuracy.

Students may select an object such as a book cover, product box, or printed marker with distinctive visual features.

3. Software and Hardware Requirements

1. Python 3.x
2. OpenCV
3. NumPy
4. Webcam or mobile camera
5. Python IDE

Install the required packages using the command:

pip install opencv-contrib-python numpy

4. Tasks to Be Completed

Task 1: Image Acquisition and Preprocessing (15 Minutes)

1. Capture a reference image of a selected object.
2. Capture a query image of the same object from a different position or angle.
3. Load both images using OpenCV.
4. Convert the images into grayscale.

Task 2: Camera Calibration Matrix (25 Minutes)

1. Understand the camera intrinsic calibration matrix.
2. Define an approximate camera matrix using the image dimensions and estimated focal length.
3. Define the lens distortion coefficients.
4. Use cv2.undistort() to correct the query image.

The camera intrinsic matrix is represented as:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Where:

- f_x, f_y: Focal lengths in pixel units.
- C_x, C_y: Principal point coordinates.
- K: Camera intrinsic calibration matrix.

Important: For meaningful calibration, students should preferably calibrate the actual camera using multiple images of a printed chessboard pattern. An approximate matrix is acceptable for this two-hour introductory exercise, but it must not be reported as a measured calibration result.

Task 3: Landmark Detection (20 Minutes)

1. Create a SIFT feature detector using OpenCV.
2. Detect keypoints in the reference and undistorted query images.
3. Extract feature descriptors.
4. Display the detected landmarks.

Task 4: Landmark Matching and Object Recognition (30 Minutes)

1. Create a Brute-Force matcher using the L2 distance metric.
2. Perform KNN matching with \(k=2\).
3. Apply Lowe's ratio test to filter weak matches.
4. Count the good matches.
5. Display the recognition result as OBJECT RECOGNIZED or OBJECT NOT RECOGNIZED.

Task 5: Testing and Documentation (30 Minutes)

1. Test the system using the same object from different viewpoints.
2. Test the system with a different object.
3. Compare results before and after image undistortion.
4. Record the number of keypoints and good matches.
5. Save the output image and prepare an observation report.

5. Python Program

Create a Python file named object_recognition_calibration.py and copy the following code.

6. Expected Output

The program should display and save the following:

1. Camera calibration matrix.
2. Number of landmarks detected in the reference image.
3. Number of landmarks detected in the query image.
4. Number of good landmark matches.
5. Object recognition result.
6. Undistorted query image.
7. Landmark matching image.

Output files:

- undistorted_query.jpg
- landmark_matches.jpg

7. Observation Table

S. No.	Test condition	Reference keypoints	Query keypoints	Good matches	Recognition result
1	Same object, similar viewpoint				
2	Same object, different viewpoint				
3	Different object				
4	Query image before/after undistortion				

8. Evaluation Scheme

9. Submission Requirements

Students must submit:

1. Python source code.
2. Reference and query images.
3. Camera calibration matrix and distortion coefficients used.
4. Saved landmark matching output.
5. Completed observation table.
6. Answers to the observation questions.

7. Extension activity: Calibrate the webcam using a chessboard pattern, estimate the camera matrix and distortion coefficients using cv2.calibrateCamera(), and use cv2.findHomography() with RANSAC to verify the object's location in the query image.